Circuit Breaker

Circuit Breaker · Protect calls to flaky rates API · Build with Waqar

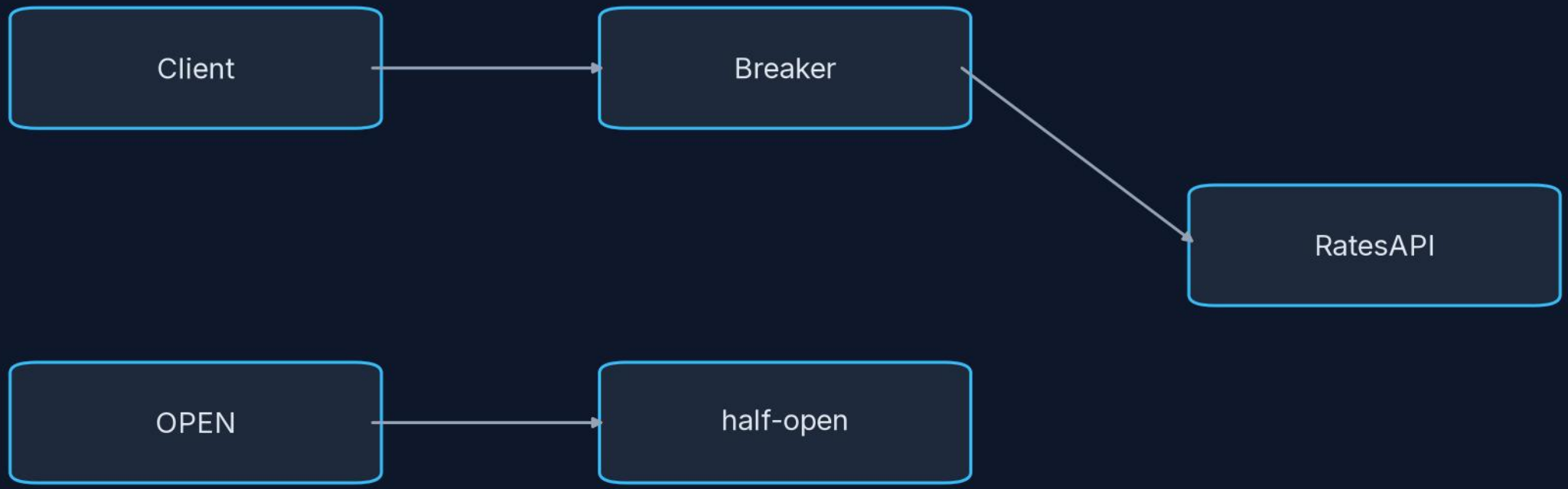
Today's Hook

- Today's force: Protect calls to flaky rates API
- We earn Circuit Breaker by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: CbState, CircuitBreaker

The Problem

- Scenario: Protect calls to flaky rates API
- Constraints: solve «Protect calls to flaky rates API» with Circuit Breaker — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Protect calls to flaky rates API
- Success looks like: fail

Structure



Circuit Breaker: Protect calls to flaky rates API

Real-World Scenario

- Running example: Protect calls to flaky rates API
- Key types: CbState, CircuitBreaker
- Happy-path stdout cue: fail
- Failure-path stdout cue: fail

Naive Code

```
package com.buildwithwaqar.patterns.enterprise.circuitbreaker.problem;

import java.util.*;

public final class NaiveRates {
  private int fails;
  public String rate(){ if(fails++ < 3) throw new RuntimeException("down"); return "83.2"; }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Protect calls to flaky rates API
- Without Circuit Breaker, call sites absorb the wrong details.
- Failure mode to show on camera: fail
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.enterprise.circuitbreaker.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

enum CbState { CLOSED, OPEN, HALF_OPEN }
final class CircuitBreaker {
  private final int threshold; private int failures; private CbState state=CbState.CLOSED; private long openedAt;
  private final long resetMs; CircuitBreaker(int threshold, long resetMs){ this.threshold=threshold; this.resetMs=resetMs; }
  CbState state(){ return state; }
  String call(Supplier<String> action){
    if (state==CbState.OPEN){
      if (System.currentTimeMillis()-openedAt < resetMs) throw new IllegalStateException("circuit open");
      state=CbState.HALF_OPEN;
    }
    try {
      String r=action.get(); failures=0; state=CbState.CLOSED; return r;
    } catch (RuntimeException e){
      failures++;
      if (failures>=threshold || state==CbState.HALF_OPEN){ state=CbState.OPEN; openedAt=System.currentTimeMillis(); }
      throw e;
    }
```

Solution (2)

```
    }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.circuitbreaker.problem;

import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Supplier;

public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Circuit Breaker: flaky rates API ===");
    System.out.println("--- naive ---");
    var n=new NaiveRates();
    for(int i=0;i<4;i++){ try{ System.out.println(n.rate()); } catch(Exception e){ System.out.println("fail"); } }
    System.out.println("--- circuit breaker ---");
    AtomicInteger blows=new AtomicInteger();
    var cb=new CircuitBreaker(2, 10);
    Supplier<String> flaky=() -> { if(blows.getAndIncrement()<2) throw new RuntimeException("down"); return "rate=83.2"; };
    for (int i=0;i<2;i++){ try{ System.out.println(cb.call(flaky)); } catch(Exception e){ System.out.println("fail state="+cb.state()); } }
    try { cb.call(flaky); } catch(IllegalStateException e){ System.out.println("shortCircuit="+e.getMessage()+" state="+cb.state()); }
    Thread.sleep(15); blows.set(100);
    System.out.println("recovered="+cb.call(() -> "rate=83.2")+" state="+cb.state());
  }
}
```

Walkthrough Output

```
=== Circuit Breaker: flaky rates API ===
--- naive ---
fail
fail
fail
83.2
--- circuit breaker ---
fail state=CLOSED
fail state=OPEN
shortCircuit=circuit open state=OPEN
recovered=rate=83.2 state=CLOSED
```

Step-by-Step Trace

- Match stdout to stages — look for: fail
- Then the injected failure: fail
- Re-run: java -cp ... 30-circuit-breaker ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Circuit Breaker
// Prefer records + sealed where they shrink the demo.
// Problem under test: Protect calls to flaky rates API
```

Trade-offs

- Use Circuit Breaker when «Protect calls to flaky rates API» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Circuit Breaker in product language.

Related Patterns

- Name the pattern usually confused with Circuit Breaker and separate the forces.
- Sketch the roles for «Protect calls to flaky rates API» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Protect calls to flaky rates API
- Naive pain → Circuit Breaker boundary → Demo stdout.
- Remember the failure cue: fail
- Break a ProblemTest assertion, watch it fail, restore it.